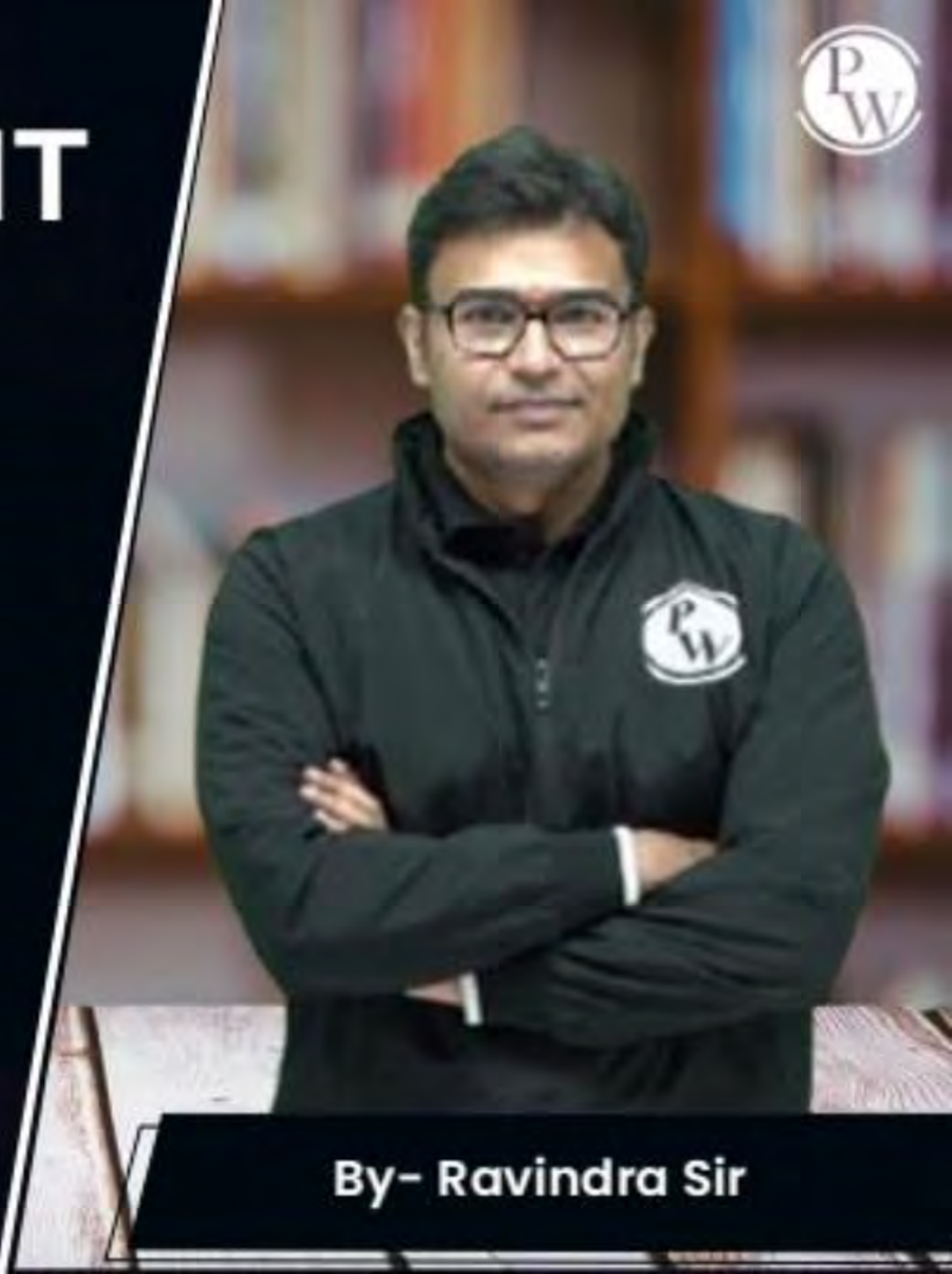
Computer Science & IT

COMPILER DESIGN
(CD)

Parsers

Lecture No. 11



By- Ravindra Sir

Recap of Previous Lecture



- Topic
- Topic
- Topic

Parsun

Topics to be Covered



- Topic
- Topic
- Topic

Parser

Join Telegram

@RBRSIRPW





Inspiring Stories : Baby Halder

Baby Halder – The Maid Who Became a Bestselling Author

Background: Child bride at 12. Mother of 3 by 20. Worked as a domestic help in Delhi to survive.

Struggles: Abusive marriage, abandoned with children. Cleaned houses during the day, raised kids, wrote at night with discarded notebooks.

Achievements: Wrote "A Life Less Ordinary", a raw memoir translated into over 20 languages. Invited to global lit fests.

Impact: She didn't have peace, privilege, or polish — just persistence. If she could write her way out of despair, you can code your way out of confusion.



Inspiring Stories : Julián Carrillo

Julián Carrillo – The Man Who Heard What No One Else Did

Background: Mexican composer born in 1875. Grew up surrounded by poverty but obsessed with sound.

Struggles: Ridiculed for claiming there were microtones between piano notes. Science rejected him. He lost positions, funding, reputation.

Achievements: Developed the "13th Sound" theory, now foundational to microtonal music. His instruments are studied even today.

Impact: Everyone thought he was hearing wrong — he was just hearing ahead of his time. If he could persist through rejection, you can persist through revision.



Inspiring Stories : Kavita Thakur

Kavita Thakur – The Kabaddi Girl from the Hills

Background: Born in a remote Himachal village, daughter of a sweeper in a local school. No shoes, no mat, no coach.

Struggles: Trained barefoot on rocky terrain, faced taunts for playing a "boy's game." The family could barely afford to travel.

Achievements: Represented India in the 2014 Asian Games, brought home Gold in Kabaddi.

Impact: From dust to podium, without ever complaining. If she could wrestle out of poverty, you can wrestle out of low ranks.

Inspiring Stories : Rajesh Sharma

Rajesh Sharma (IIT Bombay – Computer Science)

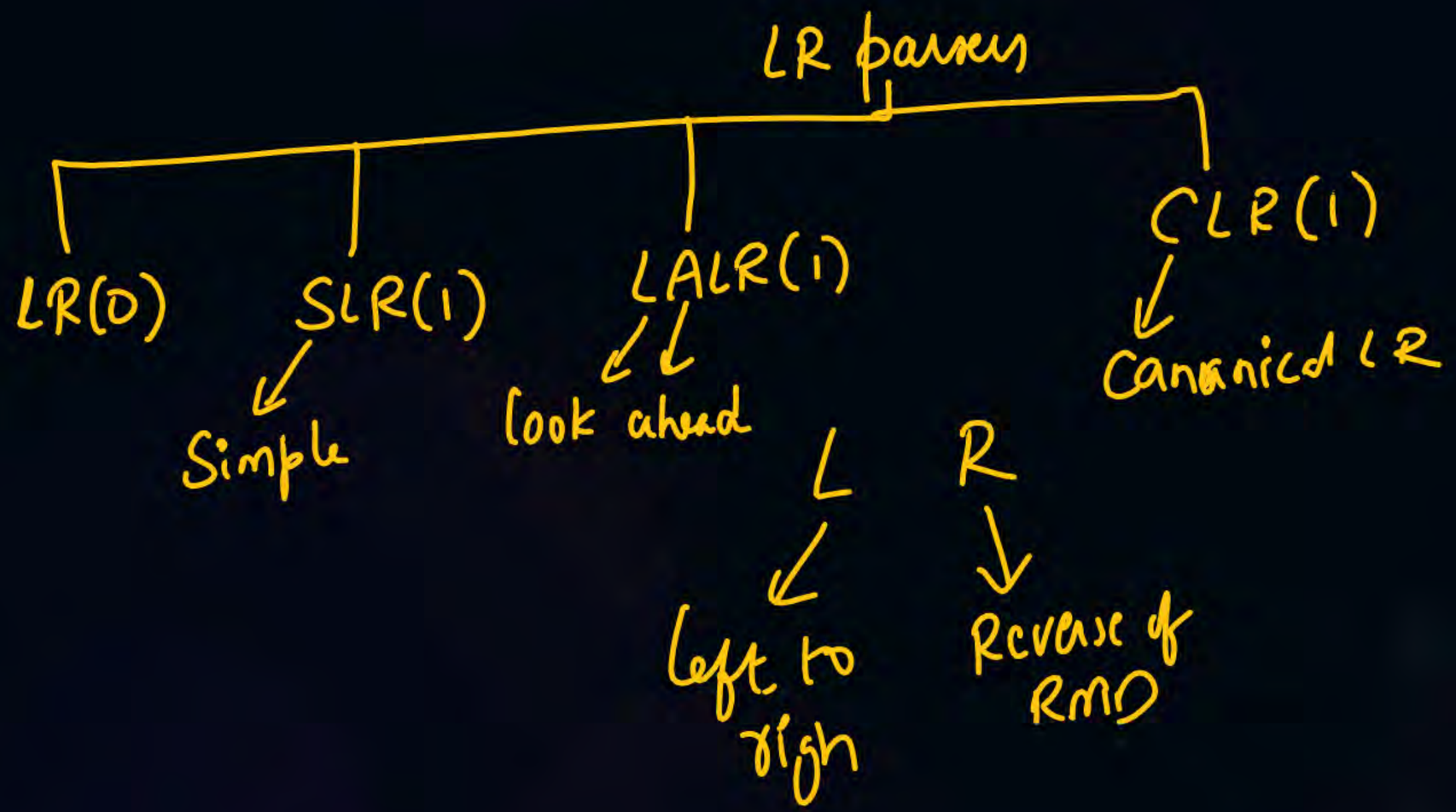
Background: **Son of a carpenter from Jodhpur; suffered from a clubfoot and couldn't afford coaching.**

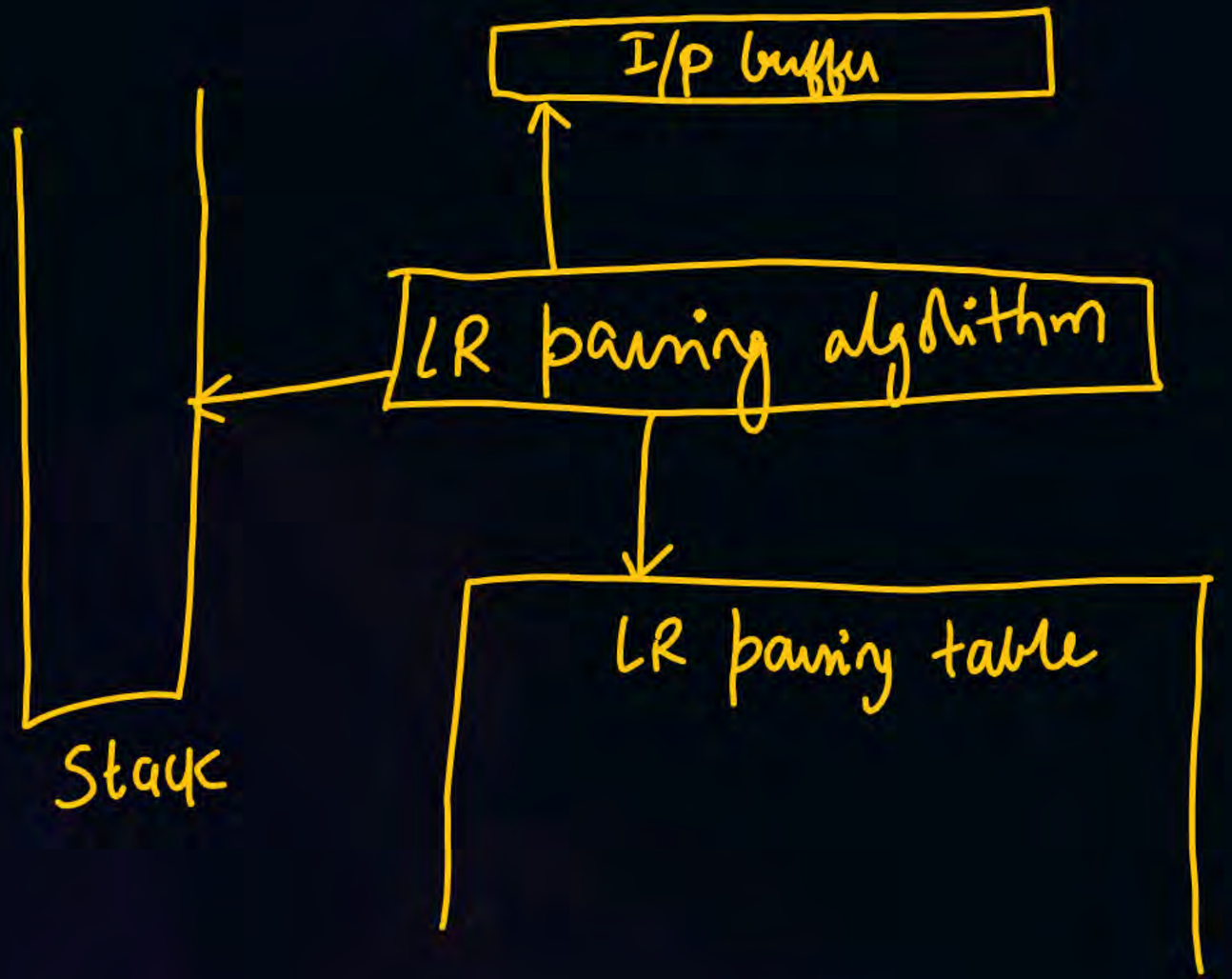
Education: **BTech from IIT Bombay, cleared JEE via self-study.**

Career Achievements: **Secured a ₹32 LPA job at Rakuten in Japan during campus placements.**

Impact: **Shattered stereotypes about disability and financial constraints, inspiring others to pursue big dreams.**

LR parsers


LR parsers
LR(0)
SLR(1)
LALR(1)
CLR(1)
Simple
look ahead
Canonical LR
L
R
Left to right
Reverse of RMD


I/p buffer
LR parsing algorithm
LR parsing table
Stack

To Construct LR(0)/SLR(1) parsing table,

1) For the given grammar, take augmented grammar
2) Create Canonical Collection of LR(0) items
3) Draw DFA with the set of items and then prepare table

To Construct CLR(1)/LALR(1) parsing tables

1) For the given grammar take augmented grammar
2) Create Canonical collection of LR(1) items
3) Draw DFA and then prepare parsing table

Augmented grammar

Given a grammar 'G' with start symbol 'S', we change the Start symbol to S' and add the rule S' → S.
The grammar with newly added production is called

augmented grammar

This rule helps the parser to understand when to stop parsing and announce the successful completion of the process.

Why is augmented production used?

In BUP, we start with i/p string and perform reductions until the string is reduced to start symbol. So reducing by start symbol is the final reduction, Sometimes there is a confusion when the start symbol may appear on the RHS of some production, because of which reduction by 'S' can be used in between not only at the end.

$S' \rightarrow S$

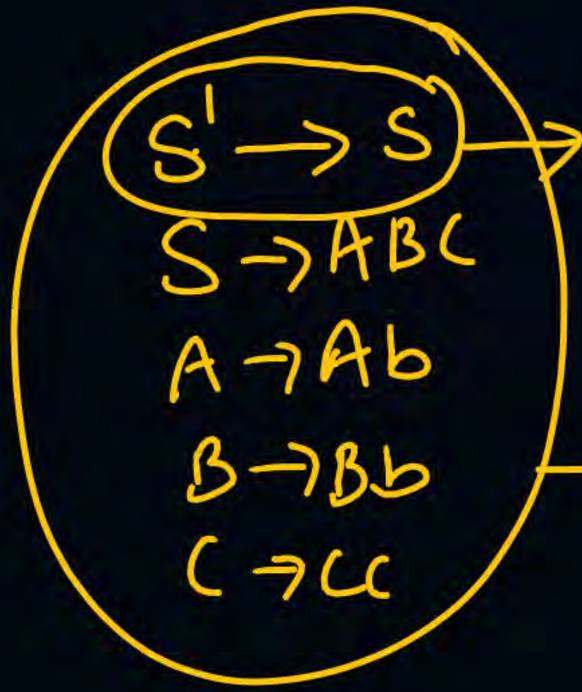
$S \rightarrow (S) / \epsilon$



$S' \to S$

$S \to aSc / ba$





→ augmented production

→ augmented grammar

LR(0) items:-

A production with dot at any point on the RHS of a production is called LR(0) item.

A bottom up parser, starts with i/p string and reads symbol by symbol from left to right until a handle is met. Handle is nothing but the RHS of any production. Once the handle is found, it reduces. So dot tells us how far the right hand side is seen by the parser.

$X \rightarrow \cdot abc$

$X \rightarrow a \cdot bc$

$X \rightarrow ab \cdot c$

$X \rightarrow abc \cdot$ → final item

Complete item

To Construct LR(0)/SLR(1) we have to create Canonical Collection of LR(0) items. To create Canonical collection of LR(0) items, we use two functions Closure() and goto()

Closure (I):- I is a set of LR(0) items like $A \to \alpha \cdot B\beta$, $C \to a \cdot$. The function closure takes as i/p set of LR(0) items and produces a set of LR(0) items as o/p.

Rules to find closure (I):-

1) initially add every item from I/p to o/p
2) If $A \to \alpha \cdot B\beta$ is in I, where B is a non terminal and $B \to \gamma$ is a rule for B, then add $B \to \cdot\gamma$ to closure (I). Repeat this for every newly added production

Goto(I, X) :-

Where I is a set of LR(0) items and X is a grammar symbol

goto(I, X) = closure ($A \to \alpha X . \beta$ / $A \to \alpha . X\beta$ is in I)

Goto involves two steps :-

1) Find transitions
2) Apply closure() on resulting items

Creating Canonical Collection 'C' of LR(0) items :-

1) The initial item in C is I_0

I_0 = closure (augmented production with dot at the beginning)

Ex :- I_0 = closure ($E' \rightarrow .E$)

2) For each I_i in C and each grammar symbol 'X' in 'G'

Repeat

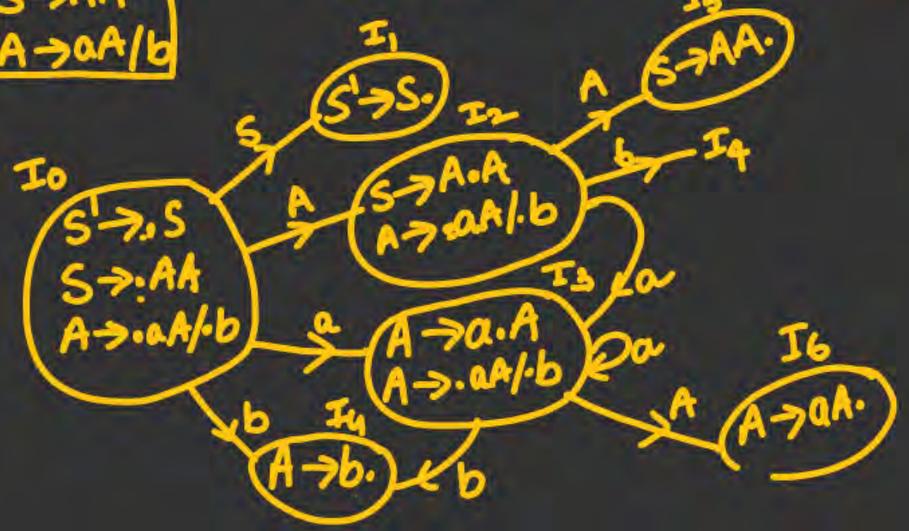
while goto(I, X) is not in Set 'C' and not a empty set

add goto(I, X) to 'C' until no more new items

Can be added to Set 'C'

S'→S
S→AA
A→aA/b

Canonical collection of LR(0) items



$S \to AA$ ①
$A \to aA$ ②
$A \to b$ ③

LR(0) parsing table

	action			GOTO	
	a	b	$	A	S
I_0	S_3	S_4		2	1
I_1			acc		
I_2	S_3	S_4		5	
I_3	S_3	S_4		6	
I_4	γ_3	γ_3	γ_3		
I_5	γ_1	γ_1	γ_1		
I_6	γ_2	γ_2	γ_2		

$S \to AA$ (1)
$A \to aA$ (2)
$A \to b$ (3)

$A \to |b|$



LR(0) parsing table

	action			GOTO	
	a	b	$	A	S
I_0	S_3	S_4		2	1
I_1			acc		
I_2	S_3	S_4		5	
I_3	S_3	S_4		6	
I_4	r_3	r_3	r_3		
I_5	r_1	r_1	r_1		
I_6	r	r_2	r_2		



LR parsing algorithm:-

Let 'X' be the state on top of the stack
and 'a' be the symbol pointed by "ip"
Action part of the parsing table is used to take parsing decisions as follows

Repeat forever:

1) If action[X,a] = S_i then (Shift action). Push 'a' then state I onto the stack. Advance i/p pointer to the next symbol
2) If action of [X,a] = r_i then (reduce action). Take i^{th} production from the grammar. Let i^{th} production be $A \to \alpha$. Pop $2 * |\alpha|$

Symbols from the stack and replace them by non terminal 'A'.
If X_i in the state below the non terminal 'A', then push
goto $[X_i, A]$ onto the stack. O/P production $A \rightarrow \beta$
}
Continue parsing
3. If action $[X, a]$ = accept then Successful completion

THANK - YOU